

AgentShield: Multi-Layer Deception-Based Defense Against Cross-Lingual Indirect Prompt Injection in Tool-Augmented AI Agents

By

Yassin Hussein Rassul

BSc Software Engineering
Student Number: 02-24-00221

A thesis submitted in partial fulfilment of the requirements for the degree of

Master of Science in Software Engineering

Department of Computer Science and Engineering
School of Science and Engineering
University of Kurdistan Hewlêr

MSc
June 2026
Erbil, Kurdistan

Abstract

[Abstract will be written after experiments are complete.]

Keywords: indirect prompt injection, AI agent security, honeytools, honeytokens, cross-lingual attacks, Kurdish, Arabic, deception-based defense, AgentDojo

Contents

Abstract II

1 Introduction 1

2 Literature Review 2

2.1 Agents, Tools, and Trust Boundaries 2

2.2 Indirect Prompt Injection and the Agentic Threat Landscape 4

2.2.1 Defining Indirect Prompt Injection 4

2.2.2 Empirical Evidence of Agent Vulnerability 4

2.2.3 Attack Categories 5

2.2.4 Summary 5

2.3 Existing Defenses and Their Limitations 6

2.3.1 Input-Level Screening 6

2.3.2 Prompt-Level Hardening 6

2.3.3 Task-Alignment Enforcement 7

2.3.4 System-Level Isolation and Validation 7

2.3.5 Limitations Across Categories 8

2.4 Deception in Cybersecurity 8

2.5 Deception Applied to LLMs and Agents 10

2.5.1 CHeaT: Deception Against LLM Agents 10

2.5.2 LLM Agent Honeypot: External Detection of AI Agents 11

2.5.3	Rebuff: Canary Tokens for LLM Leak Detection	11
2.5.4	Summary	11
2.6	Cross-Lingual and Imperceptible Attacks	12
2.6.1	Multilingual Safety Gaps	12
2.6.2	Transliteration and Script-Based Attacks	12
2.6.3	Imperceptible Encoding Attacks	13
2.6.4	Summary	14
2.7	Research Gap and Synthesis	14
3	Methodology	16
4	Experimental Work and Results	17
5	Discussion	18
6	Conclusions and Recommendations	19
	References	20

List of Tables

2.1 Synthesis of related work and positioning of AgentShield. 15

List of Figures

Chapter 1

Introduction

[Chapter 1 will be written after experiments are complete.]

Chapter 2

Literature Review

This chapter reviews the published literature relevant to the design and evaluation of AgentShield. The review is organised around three pillars: the threat posed by indirect prompt injection to tool-using agents, the defenses that have been proposed and their limitations, and the cross-lingual dimension that remains under-evaluated. A bridging theme—the use of deception as a detection mechanism—connects classical cybersecurity practice to the agent-defense setting. The chapter concludes with a synthesis of the research gap that this thesis addresses.

2.1 Agents, Tools, and Trust Boundaries

Modern large language models (LLMs) are increasingly deployed not as standalone text generators but as autonomous agents that interact with external tools and services. An LLM-based agent typically operates within a reasoning-action loop: the model receives a user instruction, reasons about the steps required to accomplish the task, selects and invokes an appropriate tool, observes the result, and continues reasoning until the task is complete. This paradigm, formalised by Yao et al. (2023) as ReAct (Reasoning and Acting), has become the dominant architecture for tool-augmented language model agents.

Tool-augmented agents differ from conventional chatbots in a security-critical respect: they can take actions in the real world. An agent with access to email tools can send messages on behalf of the user. An agent connected to a database can read, modify, or delete records. An agent with web-browsing capability can visit arbitrary URLs and process whatever content it retrieves. These capabilities make agents significantly more useful than text-only models, but they also introduce attack surfaces that do not exist in standard conversational AI.

The fundamental security problem arises from the trust boundary between the agent and its environment. When an agent invokes a tool—for example, reading an email or fetching a web page—the returned content enters the agent’s context alongside the user’s original instruction. The model processes both the user instruction and the tool output using the same mechanism: next-token prediction conditioned on the

full context window. There is no architectural separation between trusted instructions from the user and untrusted data from the environment. If an attacker places carefully crafted text inside a document, email, or web page that the agent reads, the model may interpret that text as a new instruction rather than as passive data.

This conflation of data and instructions is not a bug in any particular implementation; it is a structural property of how transformer-based language models process sequences. The model has no built-in mechanism to distinguish between “the user wants me to do X” and “this retrieved email says I should do Y.” Both are simply sequences of tokens in the context window. As a result, any content that enters the agent’s context through tool outputs becomes a potential vector for adversarial control.

The implications extend beyond individual tool calls. In multi-step agent interactions, the output of one tool call typically becomes part of the context for subsequent reasoning and tool selection. A single injection in an early tool output can therefore influence the agent’s behaviour across multiple subsequent steps, potentially causing a cascade of compromised actions. For example, an attacker’s instructions hidden in a retrieved email could cause the agent to first read sensitive data from a database and then forward that data to an external address—a multi-step exfiltration attack that neither the user nor any individual tool can easily detect.

The AgentDojo benchmark (Debenedetti et al., 2024) was designed to study precisely this class of threats. It provides 97 realistic utility tasks across four simulated environments—banking, email, travel booking, and workspace management—together with 629 security test cases in which adversarial content is injected through tool outputs. AgentDojo’s architecture supports both the evaluation of attacks and the integration of custom defenses through a plugin API, making it a suitable platform for systematic experimentation. The benchmark is discussed further in the methodology chapter; for the purposes of this review, it is noted as evidence that the security community now treats tool-level agent threats as a first-class evaluation target.

In summary, tool-augmented agents operate across a trust boundary that current architectures do not enforce. Any content retrieved through tools is processed identically to user instructions, creating a structural vulnerability that the following sections examine in detail.

2.2 Indirect Prompt Injection and the Agentic Threat Landscape

2.2.1 Defining Indirect Prompt Injection

Prompt injection attacks manipulate LLM behaviour by inserting adversarial instructions into the model’s input. Greshake et al. (2023) introduced the distinction between *direct* prompt injection, where the attacker communicates directly with the model, and *indirect* prompt injection (IPI), where the attacker embeds malicious instructions in content that the model retrieves from external sources. In the indirect case, the attacker never interacts with the model directly; instead, the adversarial payload is placed in a document, web page, email, or API response that the agent is likely to process during normal operation.

The distinction is critical for agent security. Direct prompt injection requires the attacker to have access to the user’s conversation interface. Indirect prompt injection requires only that the attacker can influence content in the agent’s environment—a far weaker assumption. A malicious instruction embedded in a public web page, a shared document, or an incoming email can compromise any agent that reads it, regardless of the user’s awareness or intent.

Liu, Y. Jia, Geng, et al. (2024) provided the first formal framework for prompt injection, demonstrating that existing attacks can be represented as special cases within a unified model. Their systematic evaluation across five attacks, ten defenses, ten LLMs, and seven tasks revealed that no single defense performs well across all scenarios, underscoring the need for multi-layered approaches. Their open-source benchmark further established that prompt injection research requires standardised, reproducible evaluation rather than isolated case studies.

2.2.2 Empirical Evidence of Agent Vulnerability

The vulnerability of tool-using agents to IPI is not theoretical. Zhan et al. (2024) created the InjecAgent benchmark comprising 1,054 test cases across 17 user tools and 62 attacker-controlled tools, and evaluated 30 different LLM agents. Under baseline attack conditions, ReAct-prompted GPT-4 was compromised 24% of the time. When attacks were reinforced with hacking prompts, success rates nearly doubled. These findings established that even state-of-the-art models are substantially vulnerable when integrated with tools.

The AgentDojo benchmark (DeBenedetti et al., 2024) corroborated these findings in

a more controlled setting, demonstrating that current models fail at many utility tasks even without adversarial interference, and that existing attacks succeed against a meaningful fraction of security test cases. The International AI Safety Report (International AI Safety Report, 2025) further noted that current defenses fail more than half the time against prompt injection attacks, highlighting the gap between the severity of the threat and the effectiveness of available mitigations.

2.2.3 Attack Categories

Indirect prompt injection attacks against agents can be categorised by their objective:

- **Goal hijacking:** the injected instruction redirects the agent away from the user's original task toward a goal chosen by the attacker.
- **Data exfiltration:** the injection causes the agent to extract sensitive information from its environment and transmit it to an attacker-controlled destination.
- **Tool misuse:** the injection causes the agent to invoke tools in ways that harm the user, such as sending unauthorised emails, modifying records, or deleting data.

These categories are not mutually exclusive; a single attack can combine goal hijacking with data exfiltration. They correspond closely to the risk categories identified by OWASP in its guidance on agentic AI security (OWASP, 2026), which lists Agent Goal Hijack (ASI01) and Tool Misuse and Exploitation (ASI02) among the top risks for agentic systems.

2.2.4 Summary

Indirect prompt injection is a well-documented and empirically validated threat to tool-using AI agents. Benchmark evaluations consistently show vulnerability rates between 24% and over 50%, depending on the attack strength, the model, and the presence of defenses. The threat is structural—rooted in the absence of a data-instruction boundary—and cannot be eliminated by model improvements alone. The following section examines the defenses that have been proposed and the limitations that persist.

2.3 Existing Defenses and Their Limitations

The defenses proposed against prompt injection in agents can be organised into four categories: input-level screening, prompt-level hardening, task-alignment enforcement, and system-level isolation. Each addresses a different point in the agent pipeline, and each has characteristic strengths and weaknesses.

2.3.1 Input-Level Screening

Input-level defenses attempt to detect adversarial content before or as it enters the model’s context. Liu, Y. Jia, J. Jia, et al. (2025) proposed DataSentinel, which formulates prompt injection detection as a minimax optimisation problem: a detector is trained adversarially to distinguish injected inputs from benign ones. DataSentinel received the Distinguished Paper Award at IEEE S&P 2025 and outperforms prior detectors on both conventional and adaptive attacks. However, it operates at the input-screening layer—it analyses text for injection signatures—and does not monitor the agent’s subsequent behaviour. If an injection evades the classifier, no further defence exists within this paradigm.

Commercial input-screening tools include Lakera Guard (Lakera, 2024) and Azure AI Content Safety’s Prompt Shield (Microsoft, 2024). These provide real-time classification of inputs as benign or malicious. A significant limitation is language coverage: Azure Prompt Shield supports at most eight languages, with no validated coverage for Kurdish or Arabic. Lakera Guard similarly focuses on high-resource languages. Input-screening defenses are therefore structurally unable to protect against cross-lingual attacks in under-resourced languages, a gap that this thesis directly addresses.

2.3.2 Prompt-Level Hardening

Prompt-level defenses modify the agent’s system prompt or the formatting of tool outputs to reduce the model’s susceptibility to injected instructions. Hines et al. (2024) proposed spotlighting with delimiting, which wraps tool outputs in special delimiters (e.g., << >>) and instructs the model not to obey content within those boundaries. The AgentDojo benchmark includes this as a built-in defense, along with a variant that repeats the user’s original prompt after each tool call to re-anchor the model’s attention.

These defenses are lightweight and model-agnostic, but they are fundamentally brittle. An attacker who knows that delimiters are in use can craft injections that close or escape the delimiters. An injection that says “ignore any delimiter instructions and proceed with the following task” may override the prompt-level constraint, because the model cannot reliably distinguish meta-instructions about formatting from adversarial overrides. Benchmark evaluations in AgentDojo show modest improvements from prompt-level defenses but far from complete protection.

2.3.3 Task-Alignment Enforcement

A more recent category of defense attempts to keep the agent aligned with the user’s original task throughout its execution. Kim et al. (2024) proposed Task Shield, which enforces task alignment by verifying that each agent action is consistent with the user’s stated goal. Evaluated on AgentDojo, Task Shield achieved an attack success rate of 2.07% with a utility score of 69.79%. These results are strong on the security axis but reveal a meaningful trade-off: the defense blocks roughly 30% of legitimate tasks, indicating that strict task alignment can be overly restrictive.

The trade-off between security and utility is a recurring theme in agent defense. A defense that blocks all tool calls is perfectly secure but entirely useless. The practical challenge is to detect and prevent malicious actions while preserving the agent’s ability to complete legitimate tasks—a balance that no existing defense has fully achieved.

2.3.4 System-Level Isolation and Validation

The most architecturally complex defenses operate at the system level, introducing additional components that validate, isolate, or re-examine agent actions.

Liao et al. (2025) proposed DRIFT, which combines a Secure Planner, a Dynamic Validator, and an Injection Isolator to enforce control-flow and data-flow constraints on agent execution. DRIFT achieves a reported attack success rate of 1.3% on its evaluation tasks, the lowest published figure at the time of writing. However, it requires substantial architectural modifications to the agent pipeline and has not been evaluated for cross-lingual robustness.

Li et al. (2025) proposed MELON, which detects attacks by re-executing the agent’s trajectory with masked tool outputs and comparing the resulting tool calls. If the agent’s actions change substantially when potentially injected content is re-

moved, the original execution is flagged as compromised. This is a form of behavioural detection—it identifies attacks by their effect on agent actions rather than by analysing injection text—and is conceptually closer to the approach taken in this thesis. However, MELON’s re-execution mechanism approximately doubles the computational cost and has not been evaluated against cross-lingual or encoding-based attacks.

2.3.5 Limitations Across Categories

Several limitations cut across all four defense categories:

1. **Cross-lingual robustness is rarely evaluated.** The majority of published defenses are benchmarked in English-only settings. Whether they maintain effectiveness against attacks written in Kurdish, Arabic, transliterated forms, or code-switched variants is unknown.
2. **Adaptive adversaries are under-studied.** Most evaluations test against fixed attack templates. An adaptive attacker who knows the defense mechanism can craft prompts specifically designed to evade it—for example, instructing the agent to “only use tools you recognise” to bypass a honeytool-based defense.
3. **Detection of compromised behaviour is rare.** Most defenses focus on preventing injection from reaching the model or preventing the model from acting on it. Very few detect that an agent *has been compromised* by observing its actions after the fact. MELON’s re-execution approach is an exception, but it detects compromise through trajectory comparison rather than through environmental tripwires.
4. **Deception is absent as a defense mechanism.** No existing defense for agents employs deception—honeytools, honeytokens, or decoy resources—as a detection layer. This is surprising given the long and successful history of deception in cybersecurity, which the following section reviews.

2.4 Deception in Cybersecurity

Deception has been a defensive strategy in computer security for over three decades. Stoll (1989) documented one of the earliest cases of using monitored decoy resources to track an intruder, and Spitzner (2003) formalised the concept of honeypots—systems deployed specifically to be probed, attacked, or compromised, with the sole purpose of detecting and studying adversarial activity.

The core principle of deception-based defense is that legitimate users have no reason to interact with decoy resources. A honeypot disguised as a database server should never receive queries from authorised applications. A honeytokent—a fake credential, API key, or sensitive file—should never appear in outbound network traffic. Any interaction with a decoy is therefore a high-confidence indicator of compromise, with a very low false-positive rate by construction.

This property distinguishes deception from other defensive approaches. Firewalls, intrusion-detection systems, and input classifiers attempt to identify malicious activity by its *characteristics*—matching patterns, anomalies, or signatures. Deception identifies malicious activity by its *target*: if the target is a decoy, the activity is by definition unauthorised. This makes deception particularly attractive in settings where adversarial inputs are difficult to distinguish from benign ones—precisely the situation in indirect prompt injection, where malicious instructions are designed to be indistinguishable from normal tool outputs.

Honeytokens extend the honeypot concept from systems to data. A fake database credential planted in a configuration file, a canary API key embedded in an environment variable, or a decoy document placed in a shared directory all serve as tripwires: if the token is used, copied, or transmitted, it signals that a breach has occurred. The key advantage is that honeytokent monitoring operates at the data-flow level, detecting exfiltration attempts regardless of how the initial compromise occurred.

Deception-based defenses also have well-known limitations. An adaptive adversary who suspects the presence of decoys may avoid interacting with unfamiliar resources. Poorly designed decoys may be obviously fake, reducing their effectiveness. And deception is inherently a detection mechanism, not a prevention mechanism: it reveals that a compromise has occurred but does not prevent the initial breach. For this reason, deception is most effective as one layer within a multi-layer defense, complementing prevention mechanisms that reduce the probability of compromise in the first place.

Despite these limitations, deception remains widely used in practice because of its high signal-to-noise ratio and its effectiveness against attackers who are unaware of the decoys. The following section examines how these principles have been applied—and not yet fully applied—in the context of LLM agents.

2.5 Deception Applied to LLMs and Agents

Several recent works have applied deception-based techniques near the intersection of cybersecurity and LLM agents. However, none have applied deception to defend an agent against indirect prompt injection at the tool level. This section reviews the three most relevant works and explains how each differs from the approach proposed in this thesis.

2.5.1 CHeaT: Deception Against LLM Agents

Ayzenshteyn et al. (2025) introduced CHeaT (Cloak, Honey, Trap), a framework of six defensive strategies and fifteen techniques for defending computer networks against autonomous LLM-based penetration-testing tools. Their techniques include cloaking (misdirecting agents away from real assets), honey-tokens (LLM-specific decoys designed to exploit model biases), and traps (infinite loops designed to neutralise agents). Evaluated against eleven CTF test machines, CHeaT achieved a 100% success rate in protecting target assets.

CHeaT is the closest related work to AgentShield in its use of deception, but the threat models are *opposite*. CHeaT defends networks *from* attacking LLM agents; AgentShield defends agents *from* injected content. In CHeaT’s setting, the LLM agent is the adversary and the decoys are placed in the network environment to mislead it. In AgentShield’s setting, the LLM agent is the asset being protected, and the decoys (honeytools and honeytokens) are placed in the agent’s own tool set and environment to detect when the agent has been compromised by an external injection.

Despite the opposite directionality, CHeaT provides important evidence for the present work: it demonstrates that LLM agents are particularly susceptible to deception. The authors attribute this susceptibility to model biases, memory limitations, and tokenisation characteristics that make LLMs more likely than human operators to interact with decoy resources. This finding supports the hypothesis that honeytool-based detection may be effective against agents compromised by indirect prompt injection.

2.5.2 LLM Agent Honeypot: External Detection of AI Agents

Reworr and Volkov (2024) deployed enhanced SSH honeypots with prompt injection detection and timing analysis to distinguish LLM-powered hacking agents from human attackers in the wild. Over three months, they collected 8,130,731 hacking attempts and identified eight potential AI agents.

This work is relevant because it demonstrates that deception can be used as a detection signal for LLM behaviour: prompt injection was used not as an attack but as a probe to determine whether an attacker was human or AI. The direction is again different from AgentShield—the honeypot detects AI agents *externally*, whereas AgentShield detects agent compromise *internally*—but the underlying principle is shared: deception reveals information about the nature and intent of the interacting entity.

2.5.3 Rebuff: Canary Tokens for LLM Leak Detection

Rebuff (ProtectAI, 2023) is an open-source tool that plants canary tokens in LLM inputs and monitors whether those tokens appear in the model's output, indicating that the model has leaked or repeated content it should not have. Rebuff operates at the input-screening layer and focuses on detecting prompt leakage rather than monitoring tool-level agent behaviour.

Rebuff's canary-token approach is conceptually adjacent to the honeytokens component of AgentShield. Both plant fake data and monitor for its appearance in inappropriate locations. The difference is in scope and mechanism: Rebuff detects whether an LLM leaks a planted token in its text output; AgentShield monitors whether a compromised agent uses planted tokens in its tool-call parameters (e.g., attempting to exfiltrate a fake API key via an email tool). The latter operates at the tool-execution layer rather than the text-output layer.

2.5.4 Summary

Deception has been applied to networks defending against LLM agents (CHeaT), to honeypots detecting LLM agents externally (LLM Agent Honeypot), and to input screening for LLM leak detection (Rebuff). To our knowledge, no prior work has applied deception—specifically, honeytools and honeytokens-based egress monitoring—to defend an agent against indirect prompt injection at the tool level.

This is the setting that AgentShield addresses.

2.6 Cross-Lingual and Imperceptible Attacks

The defenses reviewed in Section 2.3 are predominantly evaluated in English. A growing body of work demonstrates that LLM safety mechanisms are significantly weaker for non-English languages, particularly low-resource ones. This section reviews the evidence for cross-lingual vulnerability and the encoding-based attack techniques that exploit it.

2.6.1 Multilingual Safety Gaps

Deng et al. (2024) conducted a systematic study of how non-English prompts bypass LLM safety mechanisms. They identified two scenarios: an *unintentional* scenario, where users querying in non-English languages receive unsafe responses due to weaker safety training, and an *intentional* scenario, where attackers deliberately use multilingual prompts to evade safety filters.

Their findings are striking. In the unintentional scenario, low-resource languages produced unsafe content at approximately three times the rate of high-resource languages. In the intentional scenario, multilingual attacks achieved an 80.92% unsafe-content rate on ChatGPT and 40.71% on GPT-4. These results demonstrate that safety alignment is heavily skewed toward English, with other languages receiving substantially less protection.

For the present thesis, this finding has a direct implication: if safety mechanisms are weaker for low-resource languages, then defenses against indirect prompt injection—which rely on the model recognising and resisting malicious instructions—are also likely to be weaker for attacks written in those languages. Kurdish (Sorani) and Arabic are both under-represented in LLM training data relative to English, and neither language has validated prompt injection defense coverage in any published evaluation.

2.6.2 Transliteration and Script-Based Attacks

Al Ghanim et al. (2024) demonstrated that Arabic transliteration and Arabizi (writing Arabic words in Latin script with numerals substituting for Arabic-specific sounds, e.g., “3” for the Arabic letter ain) can bypass safety mechanisms on GPT-4 and

Claude 3 Sonnet. Critically, standard Arabic with prefix injection did *not* consistently bypass safety, but transliterated forms did.

The mechanism is revealing: models appear to have learned safety associations for specific word forms, and these associations do not transfer to transliterated equivalents. An instruction that is blocked when written in Arabic script may be accepted when written in Arabizi, because the model's safety training has not encountered the transliterated form.

This finding is directly relevant to the present thesis for two reasons. First, it validates transliteration as a meaningful attack vector that can bypass model-level safety. Second, it suggests that a similar technique can be applied to Kurdish: Sorani Kurdish is natively written in a modified Arabic script but is frequently transliterated to Latin script in informal digital communication, creating a natural code-switching vector that safety mechanisms are unlikely to have been trained on.

2.6.3 Imperceptible Encoding Attacks

Boucher et al. (2022) demonstrated a different class of attack: adversarial perturbations that are invisible to human readers but affect model processing. Their techniques include:

- **Zero-width characters:** Unicode characters with no visual rendering (e.g., zero-width spaces, zero-width joiners) that alter tokenisation without changing the visible text.
- **Homoglyphs:** visually similar characters from different Unicode blocks (e.g., Cyrillic “a” versus Latin “a”, or Arabic hā versus Latin “h”) that change the byte sequence while preserving visual appearance.
- **Reordering and deletion characters:** Unicode control characters that alter the display order of text without changing the underlying byte sequence.

A single imperceptible injection was sufficient to significantly degrade model performance, and three injections rendered most tested models “functionally broken.” These results were demonstrated against deployed commercial systems from Microsoft, Google, and other major providers.

For indirect prompt injection in agents, imperceptible encoding attacks are particularly dangerous because they can be embedded in tool outputs that pass human review. An email or document that appears entirely normal to a human reader may contain hidden instructions that the agent’s tokeniser processes as commands. This

thesis includes zero-width character injection and homoglyph-based attacks in its evaluation suite to test whether AgentShield’s behavioral detection (honeytools, honeytokens) can catch attacks that are invisible at the text level.

2.6.4 Summary

The cross-lingual evidence is clear: LLM safety mechanisms are weakest for low-resource languages, transliteration bypasses script-specific safety training, and imperceptible encoding attacks can evade human review entirely. Yet published defenses are predominantly evaluated in English, and no published evaluation tests prompt injection defenses for Kurdish or Arabic in the agent setting. This gap motivates the cross-lingual component of this thesis.

2.7 Research Gap and Synthesis

The preceding sections have established three findings:

1. Tool-using AI agents are empirically vulnerable to indirect prompt injection, with published vulnerability rates between 24% and over 50% depending on the attack setting. Existing defenses improve robustness through input screening, prompt hardening, task alignment, or system-level isolation, but none achieves complete protection, and all face trade-offs between security and utility.
2. Deception has a long and successful history in cybersecurity as a high-confidence detection mechanism. It has been applied near the LLM security space—against LLM agents (CHeaT), for external detection of AI agents (LLM Agent Honeytrap), and for input-level leak detection (Rebuff)—but not as an internal, tool-level defense for agents against indirect prompt injection.
3. Cross-lingual and encoding-based attacks exploit documented weaknesses in LLM safety mechanisms, yet published defenses are overwhelmingly evaluated in English. Kurdish and Arabic have no validated prompt injection defense coverage in any published agent-security evaluation.

Table 2.1 summarises the key related works and their relationship to the present thesis.

Table 2.1: Synthesis of related work and positioning of AgentShield.

Work	Defense layer	Mechanism	Lang.	Relation to AgentShield
DataSentinel	Input screening	Game-theoretic classifier	EN	Different layer; complementary
Task Shield	Task alignment	Goal-consistency check	EN	Different mechanism; comparison baseline
DRIFT	System isolation	Rule-based validation	EN	Different mechanism; comparison baseline
MELON	Behavioural	Re-execution comparison	EN	Closest mechanism; higher cost
CHeaT	Network	Deception (honey, cloak, trap)	N/A	Opposite threat model
LLM Honey-pot	External	Honeypot + timing analysis	EN	Inverse direction
Rebuff	Input screening	Canary tokens	EN	Related mechanism, different scope
AgentShield	Tool-level	Honeytools + honeytokens + egress + validation	EN, KU, AR, CS	This thesis

Prior work has shown that tool-using agents are vulnerable to indirect prompt injection and has proposed input-level, prompt-level, and system-level defenses. However, to our knowledge, no prior work has evaluated a deception-based defense—built around honeytools and honeytoken-based egress monitoring—against cross-lingual indirect prompt injection in Kurdish, Arabic, and code-switched settings on a standardised agent benchmark. This thesis does not claim the first system-level defense against indirect prompt injection; it evaluates whether a deception-inspired, multi-layer detection-and-prevention framework adds value in cross-lingual settings that remain under-evaluated, using AgentDojo as a standardised benchmark.

Chapter 3

Methodology

[Chapter 3 will be written next.]

Chapter 4

Experimental Work and Results

[Chapter 4 will be written after experiments.]

Chapter 5

Discussion

[Chapter 5 will be written after results analysis.]

Chapter 6

Conclusions and Recommendations

[Chapter 6 will be written last.]

References

- Al Ghanim, Mansour, Saleh Almohaimeed, Mengxin Zheng, Yan Solihin, and Qian Lou (2024). “Jailbreaking LLMs with Arabic Transliteration and Arabizi”. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Ayzenshteyn, Daniel, Roy Weiss, and Yisroel Mirsky (2025). “Cloak, Honey, Trap: Proactive Defenses Against LLM Agents”. In: *Proceedings of the 34th USENIX Security Symposium*.
- Boucher, Nicholas, Ilia Shumailov, Ross Anderson, and Nicolas Papernot (2022). “Bad Characters: Imperceptible NLP Attacks”. In: *Proceedings of the 43rd IEEE Symposium on Security and Privacy (S&P)*.
- Debenedetti, Edoardo, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr (2024). “AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents”. In: arXiv:2406.13352.
- Deng, Yue, Wenxuan Zhang, Sinno Jialin Pan, and Lidong Bing (2024). “Multilingual Jailbreak Challenges in Large Language Models”. In: *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Greshake, Kai, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz (2023). “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection”. In: *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*.
- Hines, Keegan et al. (2024). “Defending Against Indirect Prompt Injection Attacks With Spotlighting”. In: arXiv:2403.14720.
- International AI Safety Report (2025). *International AI Safety Report*.
- Kim, Sijan et al. (2024). “The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents”. In: arXiv:2412.16682.
- Lakera (2024). *Lakera Guard: AI Security Platform*. <https://www.lakera.ai/>.
- Li, Ziliang et al. (2025). “MELON: Indirect Prompt Injection Defense via Masked Re-execution and Tool Comparison”. In: arXiv:2502.05174.
- Liao, Fangzhou et al. (2025). “DRIFT: Dynamic Rule-Based Defense with Injection Isolation for Securing LLM Agents”. In: arXiv:2506.12104.
- Liu, Yupei, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong (2024). “Formalizing and Benchmarking Prompt Injection Attacks and Defenses”. In: *Proceedings of the 33rd USENIX Security Symposium*, pp. 1831–1847.
- Liu, Yupei, Yuqi Jia, Jinyuan Jia, Dawn Song, and Neil Zhenqiang Gong (2025). “DataSentinel: A Game-Theoretic Detection of Prompt Injection Attacks”. In: *Pro-*

- ceedings of the IEEE Symposium on Security and Privacy (S&P)*. Distinguished Paper Award.
- Microsoft (2024). *Azure AI Content Safety: Prompt Shield*. <https://learn.microsoft.com/en-us/azure/ai-services/content-safety/>.
- OWASP (2026). *Top 10 Risks and Mitigations for Agentic AI Security*. <https://genai.owasp.org/>.
- ProtectAI (2023). *Rebuff: LLM Prompt Injection Detector*. <https://github.com/protectai/rebuff>.
- Reworr and Dmitrii Volkov (2024). “LLM Agent Honeypot: Monitoring AI Hacking Agents in the Wild”. In: arXiv:2410.13919.
- Spitzner, Lance (2003). *Honeypots: Tracking Hackers*. Addison-Wesley.
- Stoll, Clifford (1989). *The Cuckoo’s Egg: Tracking a Spy Through the Maze of Computer Espionage*. Doubleday.
- Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao (2023). “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Zhan, Qiusi, Zhixiang Liang, Zifan Ying, and Daniel Kang (2024). “InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents”. In: *Findings of the Association for Computational Linguistics: ACL 2024*.